

SKCC: Portable and Secure Skill Compilation for Cross-Framework LLM Agents

Yipeng Ouyang
Sun Yat-sen University
ouyyp5@mail2.sysu.edu.cn

Yi Xiao
Sun Yat-sen University
xiaoy398@mail2.sysu.edu.cn

Yuhao Gu
Sun Yat-sen University
guyh9@mail2.sysu.edu.cn

Xianwei Zhang
Sun Yat-sen University
zhangxw79@mail.sysu.edu.cn

 <https://github.com/Nexa-Language/Skill-Compiler>
 <https://skcc.nexa-lang.com/>

Abstract

LLM agents increasingly rely on reusable skills (e.g., SKILL.md) to execute complex tasks, yet these artifacts lack portability: agent frameworks are highly sensitive to prompt formatting, leading to a large performance variation for the same skill. Nevertheless, most skills are authored once as format-agnostic Markdown, necessitating costly per-framework rewrites and also leaving security largely unaddressed, with widespread vulnerabilities in practice. To address this, we present SKCC, a compiler for LLM agents that introduces classical compilation design into agent skill development. SKCC centers on SKIR, a strongly-typed intermediate representation that decouples skill semantics from framework-specific formatting, thus enabling portable deployment across agent frameworks. Atop of this IR, a static Optimizer enforces security constraints, blocking vulnerabilities before deployment. Implemented as a four-phase pipeline, SKCC effectively reduces adaptation complexity from $O(m \times n)$ to $O(m + n)$ across m skills and n frameworks. Experiments on SkillsBench demonstrate that SKCC delivers consistent and substantial gains over original counterparts, with pass rate increases from 21.1% to 33.3% on Claude Code and from 35.1% to 48.7% on Kimi CLI. Further, the design achieves sub-10ms compilation latency, 94.8% proactive security trigger rate, and 10–46% runtime token savings across frameworks.

1 Introduction

The rapid advancement of large language models (LLMs) has catalyzed a new generation of autonomous agent systems [1–3]. Agent frameworks such as Anthropic Claude Code [4], OpenAI Codex [5], Google Gemini CLI [6], and Kimi CLI [7] provide terminal-based agent environments where LLMs interact with tools, file systems, and external services. Skills, structured prompt artifacts following the SKILL.md specification [8], have emerged as the *de facto* standard for encoding domain-specific knowledge, employing progressive disclosure [9] that loads lightweight metadata at initialization and retrieves full content on demand. As the ecosystem matures, the number of community-contributed skills has grown rapidly, with repositories such as Anthropic-skills [10], ecc-skills [11], and sentry-skills [12] collectively hosting thousands of reusable skill artifacts.

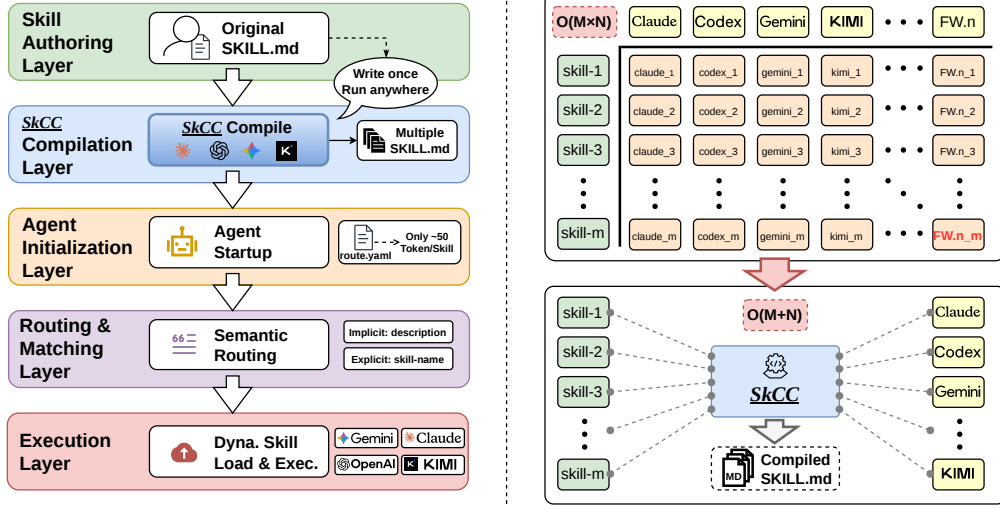


Figure 1: Left: Agent workflow with SKCC integration. Skills are authored once as SKILL.md, compiled to framework-native formats, and loaded via progressive routing manifests at agent initialization. Right: Adaptation complexity reduction from $O(m \times n)$ to $O(m + n)$. Traditional per-framework rewriting requires $m \times n$ manual adaptations. SKCC decouples skills and frameworks through a unified IR, requiring only m skill sources and n Emitter implementations.

However, a growing body of evidence reveals that LLM performance is highly sensitive to the structural format in which skills are presented [13]. For example, Claude performs substantially better when skills use XML semantic layering [14], GPT-series models benefit from XML-tagged Markdown that avoids the "format tax" of JSON [15], and deeply nested data is parsed most accurately in YAML [16]. Yet the current ecosystem assumes format-agnostic delivery: the same SKILL.md is deployed identically across all frameworks, ignoring these well-documented format preferences. The same skill can exhibit a large performance variation depending solely on how it is formatted for a given model. Beyond format compatibility, the skill ecosystem faces an equally pressing security challenge. Snyk’s audit [17] found that over one third community skills contain security vulnerabilities, including many confirmed malicious payloads. The SKILL.md specification acknowledges the need for negative boundaries [18], yet most existing skills lack such constraints, and no systematic mechanism exists to enforce security properties before skills reach an agent’s context window. These two challenges, format sensitivity and security vulnerability, are not independent. They both stem from the fundamental assumption that a single, static Markdown file can serve all frameworks and all threat models simultaneously.

We present SKCC, a systematic skill compilation design that addresses both the portability and security challenges of cross-framework skill deployment. The central insight is that a unified intermediate representation, SKIR, can decouple skill authoring from framework-specific formatting, enabling each skill to be written once and compiled to multiple frameworks. This mirrors the classical compiler architecture that just as LLVM IR enabled a single frontend to target diverse hardware backends, SKIR enables a single skill source to target diverse agent frameworks. SKCC operates through a four-phase pipeline: ①a Syntax Parser extracts AST from raw SKILL.md, ②an IR Builder transforms it into a strongly-typed SKIR, ③a Security Optimizer enforces safety constraints via Anti-Skill Injection, and ④a polymorphic Target Emitter renders the validated IR into framework-native formats. This architecture reduces the adaptation complexity from $O(m \times n)$ to $O(m + n)$.

Our key contributions are as follows:

- **We identify a structural gap in the agent skill ecosystem:** format sensitivity is a first-class concern in skill deployment, and the growing diversity of agent frameworks makes manual per-framework adaptation infeasible, motivating a compiler-based solution with a unified intermediate representation.
- **We propose SKCC,** a four-phase skill compilation design that achieves portable deployment via SKIR, and secure execution through Anti-Skill Injection and semantic validation. By

introducing a unified IR and a polymorphic emission layer, SKCC decouples skill authoring from framework-specific formatting, reducing the $O(m \times n)$ adaptation burden to $O(m + n)$ while enforcing security constraints before deployment.

- **We implement and evaluate SKCC across mainstream agent frameworks**, demonstrating consistent pass rate improvements (up to +13.5%), sub-10ms compilation latency, 94.8% Anti-Skill Injection coverage, and 10–46% runtime token savings, demonstrating strong gains in portability, security, and efficiency across frameworks.

2 Background and Motivation

2.1 Background

Our Design covers two areas: how agent skills are structured and used in practice, and the classical compilation principles that inform our approach.

Agent Skills: Structure and Usage. Modern LLM agent systems [1–3] execute complex tasks by composing tool calls, file system operations, and external service interactions. To encode domain-specific knowledge in a reusable form, the community has converged on SKILL.md [8], a portable specification consisting of YAML frontmatter for metadata and a Markdown body for executable instructions. Skills are loaded through progressive disclosure [9]: a lightweight routing manifest (~ 50 tokens per skill) is loaded at initialization, and full content is retrieved on demand when semantically matched to the user’s task. Skills interact with external systems through the Model Context Protocol (MCP) [19], a standardized interface for connecting agents to tools and services. Agent skills and their MCP interactions rely on several structured data formats: XML (tag-delimited trees), JSON (key-value pairs), YAML (indentation-based nesting, superior LLM parsing accuracy for deeply nested structures [16]), and Markdown (lightweight markup). These syntactic differences directly affect how accurately LLMs extract and follow instructions.

Classical Compilation Principles. A traditional compiler [20, 21] transforms source code through a multi-phase pipeline: lexical analysis, parsing into an AST, semantic analysis, IR generation, optimization, and target code generation. The critical architectural insight is the role of the IR: by introducing a unified intermediate layer, compilers decouple frontend language parsing from backend code generation, reducing the $m \times n$ support problem to $m + n$ [22–24]. Security optimization at compile time, such as stack canary insertion and bounds checking [25], further demonstrates that compilers can enforce safety properties before code executes.

2.2 Related Work and Challenges

Having established the foundational concepts, we now analyze recent work and identify limitations that motivate our approach.

Format Sensitivity and Skill Retrieval. LLM performance is highly sensitive to prompt formatting, with up to 40% variation from format changes alone [13]. Framework-specific preferences are well-documented: Claude benefits from XML semantic layering [14, 26, 27], GPT-series models suffer from a “format tax” with JSON [15, 28], and YAML achieves superior parsing accuracy for nested data [16]. CFPO [29] jointly optimizes content and format through iterative refinement, but its search-based approach is computationally expensive and produces instance-specific rather than reusable rules. On the retrieval side, recent work explores generation, augmentation, graph, and embedding skill retrieval [30–33], and Liu et al. [34] show that query-specific refinement yields modest post-retrieval gains. These works share a common assumption, that skills once retrieved are format-agnostic and require no structural adaptation.

Compilation and Security for Agent Skills. Applying compilation techniques to LLM systems has gained traction: Mikek et al. [35] demonstrate compiler-LLM cooperation for agentic code optimization, and Kim et al. [36] use compiler orchestration for parallel function calling. SkVM [37] also explores compilation concepts for agent skills with a JVM-like architecture supporting capability profiling and AOT/JIT optimization. On the security dimension, Snyk’s audit [17] finds 37% of 3,984 community skills contain vulnerabilities, yet the SKILL.md specification’s recommended negative boundaries [18] are rarely followed [38], and recent work on secure code generation [39] operates at the code rather than skill level.

Table 1: Capability Coverage of Related Systems

Method	Format Adapt.	Security	Multi-Framework	Complexity
SkVM [37]	Semantic only	×	✓	$O(m \times n)$
CFPO [29]	Iterative	×	×	$O(k \times m)$
Wild Retrieval [34]	Query-specific	×	✓	$O(m \times n)$
Desired	IR-driven	✓	✓	$O(m + n)$

Challenges. The preceding analysis reveals a structural gap: existing systems either ignore format sensitivity, address it through expensive instance-specific search, or focus on semantic capability without format-syntax adaptation, while, to our knowledge, no system provides systematic compile-time security enforcement for agent skills (Table 1).

These challenges share a common architectural root. Supporting diverse skills across diverse frameworks requires a decoupling layer, a unified intermediate representation that separates skill semantics from framework-specific formatting, combined with compile-time analysis that enforces security constraints before deployment. Rather than treating skills as static text files that must be manually rewritten for each target, a compiler-based methodology treats them as compilable artifacts: authored once in a canonical form, analyzed and optimized at compile time, and emitted into framework-native formats through platform-specific backends. This separation of concerns mirrors the classical compiler architecture that revolutionized systems programming, and we argue it is equally necessary for the emerging agent skill ecosystem.

3 SKCC Design

SKCC is a compilation pipeline that accepts a single SKILL.md source and produces framework-native skill artifacts through four phases (Figure 2). Phases 1–2 (Syntax Parser and IR Builder, §3.1) extract structured, typed representations from raw Markdown, producing SKIR, a unified intermediate representation that decouples skill semantics from framework-specific formatting. Phase 3 (Security Optimizer, §3.2) optimizes the IR through a chain of compile-time analyses that validate structure, audit permissions, inject safety constraints, and assign security levels. Phase 4 (Target Emitter, §3.3) renders the optimized IR into framework-native formats through a polymorphic emission layer. The critical architectural property is that Phases 1–3 execute once per skill; the resulting optimized SKIR is then shared across all emission targets, reducing the adaptation complexity from $O(m \times n)$ to $O(m + n)$.

3.1 Syntax Parsing and IR Building

Syntax Parser. Raw SKILL.md files interleave structural metadata (YAML frontmatter) with free-form instructional text (Markdown body), creating ambiguity for downstream consumers. The Syntax Parser eliminates this ambiguity by aggressively separating concerns at the syntactic level: metadata is deserialized into a typed routing table, while the Markdown body is lowered into a deterministic abstract syntax tree where procedure steps, code blocks, and examples are explicitly classified. This separation ensures that every subsequent phase operates on structured, unambiguous data rather than raw text, and it enables the compiler to reason about skill structure independently of authoring style.

IR Builder. The IR Builder transforms the raw AST into SKIR, a strongly-typed intermediate representation. The key methodological decision is to normalize heterogeneous skill content into a uniform, typed structure that captures what a skill *means* independently of how it is *formatted*. Rather than preserving Markdown-level details, SKIR abstracts skill information into semantic categories (procedures, permissions, schemas, constraints), each with well-defined types and validation rules. This abstraction serves two purposes. First, it provides a single source of truth that all downstream phases can consume without re-parsing or re-interpreting the original text. Second, it creates a clean boundary between skill authoring and skill deployment: authors write in a single canonical format, and the IR insulates them from the formatting requirements of individual frameworks. A concrete SKIR instance is provided in Appendix C.3.

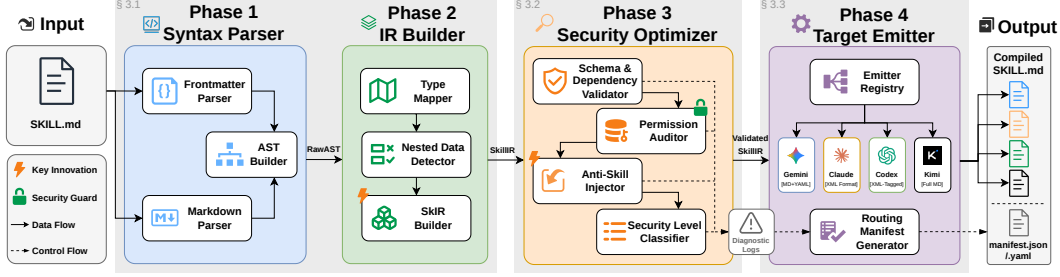


Figure 2: SKCC’s four-phase compilation pipeline. A unified SKILL.md source is parsed into a raw AST (Syntax Parser), transformed into a strongly-typed SKIR (IR Builder), validated and optimized by compile-time security analysis (Security Optimizer), and emitted into framework-native formats (Target Emitters).

A representative capability of the IR level is nested data detection: when a skill declares schemas with nesting depth exceeding a threshold, the IR records a flag that downstream Target Emitters consult to decide whether to render structured data in a format suited for deep nesting. This illustrates the IR’s broader role as an information bridge that captures semantic properties once and communicates them to every emission target without duplication.

3.2 Security Optimization

The Security Optimizer hardens the SKIR through a chain of four analyses executed in a fixed logical order. The design reflects a broader architectural philosophy: security analysis at the IR level is simultaneously format-agnostic and format-preserving. It is format-agnostic because the Optimizer operates on typed semantic structures rather than raw syntax, so a single analysis applies to all target frameworks. It is format-preserving because injected constraints are embedded in the IR itself, guaranteeing they appear in every emitted artifact regardless of the target format. This dual property is what makes compile-time security optimization both universal and reliable. Each step builds on the guarantees established by the previous one: structural validity enables meaningful permission checking, which informs constraint injection, which determines the final security classification. Together they form a defense-in-depth pipeline that transforms an untrusted skill into a validated, constrained artifact before it reaches any agent’s context window.

Structural Validation. Before any semantic analysis can be meaningful, the skill must be structurally well-formed. This step verifies that the skill’s name, description, version, and schema declarations satisfy baseline integrity constraints, and that all declared MCP dependencies resolve to known, trusted servers. Skills that fail structural validation are rejected at compile time, which is the fail-fast design that prevents malformed skills from causing unpredictable failures during agent execution.

Permission Auditing. Once structural integrity is confirmed, the Optimizer audits the skill’s declared permissions against a security baseline. It identifies overly broad access grants (e.g., unrestricted network access, filesystem writes outside allowed directories) and flags permissions that are incompatible with the skill’s stated security expectations. This step transforms permissions from passive declarations into actively enforced guardrails: skills that request dangerous capabilities must justify them through explicit, auditable declarations, and the compiler surfaces discrepancies before deployment.

Anti-Skill Injection. This is the core security mechanism of SKCC. Rather than depending on skill authors to manually embed defensive constraints (an approach that audits show fails in practice), the Optimizer automatically scans procedure text for dangerous patterns and injects corresponding safety constraints directly into the SKIR. The key design insight is that safety constraints should be a property of the compilation process, not of individual author diligence. By operating at the IR level, injected constraints become part of the skill’s semantic definition: they survive format translation and appear consistently across all target frameworks. The injection rules target common vulnerability classes (unsafe HTTP calls, unbounded loops, destructive database operations, fragile

Table 2: Example Agent Frameworks, Models, and Emission Strategies

Framework	Ex. Model	Emitter Format	Key Strategy
Claude Code	claude-opus-4-6	XML Semantic Layering	Tag-wrapped structure, up to 23% gain
Codex CLI	gpt-5.3-codex	XML-Tagged Markdown	Structural markers, avoids format tax
Gemini CLI	gemini-2.5-pro	Markdown + Conditional YAML	YAML at depth ≥ 3 (51.9% vs 43.1%)
Kimi CLI	kimi-k2.5	Full Markdown Preservation	No truncation, ultra-long context

HTML parsing), and the complete rule table is provided in Appendix C.4. Because injection happens at compile time, safety guarantees are established before the skill ever enters an agent’s context window, a fundamentally different threat model from runtime guardrails that rely on the agent’s own judgment.

Security Classification. The final step assigns each skill a tiered security level based on its accumulated analysis results. The classification enables graduated enforcement: low-risk skills proceed with minimal overhead, medium-risk skills receive passive warnings, high-risk skills require mandatory human-in-the-loop confirmation, and critical-risk skills are blocked from automatic execution entirely. This tiered design avoids a one-size-fits-all security posture: it imposes friction proportional to risk, ensuring that safe skills remain lightweight while dangerous skills are contained.

3.3 Target Emission

The Target Emitter renders the optimized SKIR into framework-native skill artifacts. Its design addresses a fundamental tension: every agent framework has distinct format preferences rooted in its underlying model’s training distribution, yet skill authors cannot reasonably be expected to master or maintain format-specific variants for every target. The Emitter resolves this tension through polymorphic emission, a single abstract interface for rendering SKIR to text, with concrete implementations that each encode the format strategy appropriate for one target framework.

The Emitter addresses three concerns that generalize across all targets. **Format Alignment** maps SKIR semantic categories to the syntactic constructs that each target framework parses most accurately, guided by the empirical format sensitivity findings discussed in §2.2. **Routing Manifest Generation** produces a lightweight index containing only the name, description, security level, and human-in-the-loop flag for each skill, enabling efficient semantic routing at agent initialization without loading full skill content, a direct implementation of the progressive disclosure pattern [8, 9]. **Token Optimization** consults IR-level flags set during earlier phases to make format decisions that reduce downstream token consumption, such as conditionally selecting a more compact representation for deeply nested data.

Table 2 summarizes four example Target Emitters evaluated in this paper; additional frameworks are supported by implementing the same Emitter interface. Detailed output examples are provided in Appendix C.5, and implementation details appear in Appendix A. The polymorphic design guarantees extensibility: supporting a new agent framework requires only a new Emitter implementation, with no changes to the prior three phases. This is the architectural property that delivers the $O(m + n)$ complexity bound: m skills pass through the shared frontend once, and n Emitters consume the same optimized SKIR.

4 Evaluation

We evaluate SkCC along three axes: (1) portability and security of SkCC-compiled skills versus format-agnostic baselines, including comparison with state-of-the-art alternatives; (2) whether compilation gains are model-specific via ablation experiments; and (3) supplementary engineering properties including compilation latency and token/time efficiency.

4.1 Experiment Setup

Benchmark and Datasets. SkillsBench [40] provides 89 real-world tasks with Docker-based execution and automated pytest verification, classified by difficulty and category. We use Pass@1 (reward

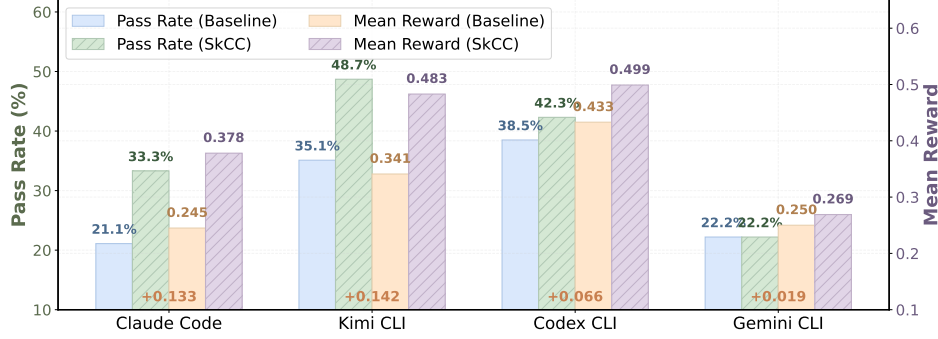


Figure 3: Pass rate and mean reward comparison of Baseline vs. SKCC-Compiled. Each framework shows four bars: Pass Rate (Baseline/SKCC) and Mean Reward (Baseline/SKCC). SKCC consistently outperforms Baseline across all frameworks and both metrics.

≥ 0.5) as our primary metric, where reward is a continuous score in $[0, 1]$ assigned by an LLM judge evaluating task completion correctness. For compilation performance and token efficiency experiments, we collected 225 skills from four community repositories: Anthropic-skills [10], ecc-skills (everything-claude-code) [11], sentry-skills (Sentry team) [12], and ui-skill [41]. Data validity details are provided in Appendix B.

LLM Models and Agent Frameworks. Table 2 in §3 summarizes the four mainstream agent frameworks, their corresponding models, and the emission strategies employed by SKCC. All experiments use the Harbor framework [42] for Docker-based task execution, with each agent framework running within Harbor-managed containers. Ablation experiments additionally test glm-5.1 and deepseek-v4-flash via the OpenHands SDK [43] integrated within Harbor.

Methods Compared. We compare **Baseline** (a single, format-agnostic SKILL.md deployed identically across all frameworks) against **SKCC** (the SKCC-compiled SKILL.md with framework-specific formatting and Anti-Skill constraints). We additionally compare against the retrieval-based refinement of Liu et al. [34] and the SkVM compilation architecture [37] in §4.2.

Metrics. We evaluate across three categories of metrics. For portability and security performance (§4.2), we measure Pass@1 (task pass rate), Mean Reward, Anti-Skill Injection trigger rate, and compilation interception counts. For ablation experiments (§4.3), we use Pass@1 and paired statistical tests (paired t-test, Cohen’s d) to quantify the effect of a fixed compiled format across different models. For supplementary properties (§4.4), we measure per-skill compilation latency (ms) and token/time efficiency during agent execution.

4.2 Overall Performance

This section evaluates SKCC’s portability and security performance.

4.2.1 Portability: Pass Rate and Mean Reward

Figure 3 reports the pass rate and mean reward for Baseline and SKCC conditions across all four frameworks.

Overall Gains. The core finding is that **SKCC compilation improves pass rate and mean reward on every tested framework**. Across all four frameworks, the SKCC condition achieves higher pass rates and mean rewards than the Baseline condition. The average pass rate improvement across frameworks is +7.0 percentage points, with the largest absolute gain on Kimi CLI (+13.5pp, from 35.1% to 48.7%) and the largest relative gain on Claude Code (+12.2pp, a 58% relative improvement). These results clearly validate SKCC’s portability claim that by decoupling skill semantics from framework-specific formatting through SKIR, a single skill source can be compiled into portable artifacts that outperform format-agnostic baselines on every target. The mechanism behind these gains is consistent: compilation transforms tasks that fail under the Baseline condition into successes. On Claude Code, 6 of 7 tasks where SKCC outperforms Baseline flipped from reward=0 to reward=1; on Kimi CLI, 13 tasks flipped from complete failure to complete success. This pattern of flipping failures rather

than incrementally improving successes is the primary source of compilation’s value. The complete four-model summary table is provided in Appendix D.1.

Framework-Specific Patterns. While compilation is broadly beneficial, the magnitude of gains varies substantially across frameworks. Claude Code and Kimi CLI show the largest improvements because both are highly format-sensitive: Claude’s training distribution heavily favors XML-tagged inputs, and Kimi’s ultra-long context window benefits from full-detail preservation without truncation. In contrast, Codex CLI and Gemini CLI show more modest gains: Codex benefits moderately from structural markers, while Gemini is relatively format-tolerant and YAML optimization only activates for deeply nested schemas. This result is not a weakness; it demonstrates that SKCC does not harm performance even on format-tolerant models, while providing security hardening as an unconditional benefit. Complete per-framework statistical test results are provided in Appendix D.2 and Appendix D.3.

Comparison with State-of-the-Art. On both Claude Code and Kimi CLI, SKCC achieves substantially larger pass rate improvements than retrieval-based refinement, and its average relative improvement (+26.6%) exceeds both Liu et al. (+20.6%) and SkVM (+15.3%). These results highlight a fundamental difference: retrieval-based refinement operates on already-retrieved skills and yields modest improvements, while SKCC’s compilation transforms the format-agnostic baseline through structural alignment with model-specific training distributions. SkVM [37] also explored compilation concepts for agent skills; however, it focuses on semantic capability profiling rather than format-syntax adaptation and does not include security hardening. Table 1 in §2 provides a structured comparison across all dimensions.

Table 3: Pass rate improvement (percentage points) over baseline. B/L = baseline, Optm. = optimized, pp = percentage points. SKCC delivers larger gains than retrieval-based refinement on both comparable model-framework pairs.

Method	Model	B/L → Optm.	Δ
SKCC	Claude	21.1% → 33.3%	+12.2pp
Liu et al.	Claude	40.1% → 48.2%	+8.1pp
SKCC	Kimi	35.1% → 48.7%	+13.5pp
Liu et al.	Kimi	19.8% → 23.1%	+3.3pp

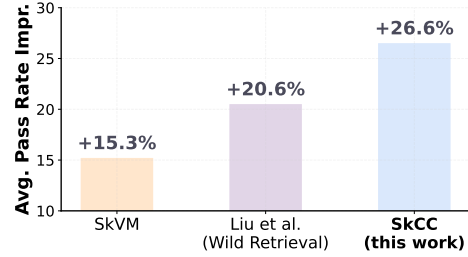


Figure 4: Average relative pass rate improvement across methods.

4.2.2 Security: Injection Trigger and Compilation Interception

Table 4: Rule trigger distribution

Anti-Skill Rule	Triggered Skills
HTTP safety	212 (91.4%)
Loop safety	104 (44.6%)
DB safety	78 (33.5%)
Parse safety	2 (0.9%)

Table 5: Compilation latency (ms) by complexity

Complexity	n	Avg	Min	Max
Simple	8	8.54	6.90	11.73
Medium	74	8.58	6.28	17.70
Complex	143	9.13	5.85	22.89
Overall	225	8.93	5.85	22.89

SKCC’s compile-time safety checking automatically detects dangerous patterns in skill content and injects protective constraints. Across 233 evaluated skills, Anti-Skill Injection triggered in **221 (94.8%)** skills, with only 12 (5.2%) skills not triggering any rule. Rule overlap is common: many skills trigger multiple rules simultaneously (HTTP + Loop + DB = most common combination, Table 4). The complete trigger statistics and rule distribution tables are provided in Appendix D.8 and Appendix D.9.

We also compile all 231 SkillsBench skills targeting the Gemini framework. 221 of 231 skills (95.7%) compile successfully, while 10 skills are intercepted by the compiler’s safety checks across three categories: YAML format violations (5 cases), security check interceptions (4 cases), and schema validation interceptions (1 case). The complete interception type table is provided in Appendix D.10. Rather than a system limitation, these interceptions highlight the efficacy of SKCC’s fail-fast design: by intercepting malformed or dangerous skills at compile time, the system prevents them from

polluting the agent’s context window or causing unpredictable runtime errors. This compile-time safety guarantee distinguishes SKCC from runtime-only safety mechanisms that rely on the agent’s own judgment, an approach that is inherently unreliable given LLMs’ tendency to follow instructions literally [17].

4.3 Ablation Study: Format Specificity

To validate that SKCC’s compiled output format is model-specific rather than universally beneficial, we conduct ablation experiments using the same Kimi-compiled output (Full Markdown) on three different models: kimi-k2.5, glm-5.1, and deepseek-v4-flash. All three experiments use the OpenHands SDK as the agent framework, with the Kimi backend format held constant. The complete ablation table with all metrics is provided in Appendix D.4.

The same compiled output produces dramatically different results across models. On Kimi, the Kimi-compiled format yields a significant positive effect ($d = +0.33$, $p = 0.0063$). On GLM-5, the effect is essentially neutral ($d = -0.03$, $p = 0.857$). On DeepSeek-v4-flash, the effect is slightly negative ($d = -0.14$, $p = 0.2561$), though not statistically significant. These results demonstrate that compilation gains are model-dependent with no one-size-fits-all optimal format, providing empirical justification for SKCC’s multi-backend architecture.

4.4 Supplementary Performance and Efficiency

All skills compile in under 10ms on average (Table 5); detailed compilation latency data is provided in Appendix A.1. We focus here on the token and time efficiency of compiled skills during agent execution.

4.4.1 Token and Time Efficiency

Real Token and Time Consumption. We now examine the token-level and temporal efficiency of SKCC-compiled skills during agent execution. The overall result is clear: SKCC-compiled skills consistently reduce both token consumption and execution time across all frameworks. On Claude Code, the SKCC condition achieves substantially lower per-task token consumption while simultaneously obtaining higher reward, demonstrating that SKCC improves task performance and token efficiency jointly. Across frameworks, compilation reduces total tokens by 10–23% and execution time by 23–43%. The complete token consumption table is provided in Appendix D.7.

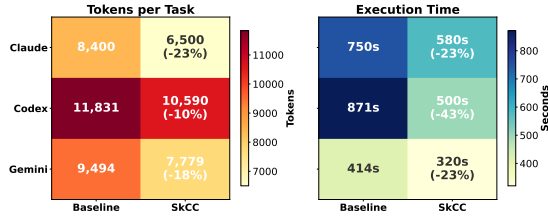


Figure 5: Cross-framework token and time efficiency heatmap. SKCC skills show consistent reductions in total tokens and execution time across all frameworks. Claude token counts are reported in hundreds due to API measurement differences.

Static Expansion vs. Dynamic Efficiency. Compilation introduces static structural overhead from XML tags, Anti-Skill constraints, and format hardening (ranging from +4% on Kimi to +25% on Claude), but this overhead is more than offset by dynamic efficiency gains during execution. Structured formats serve as cognitive scaffolding that reduces model trial-and-error and redundant output, yielding net token savings of 10–46% across frameworks. The complete expansion overhead table by complexity is provided in Appendix D.6.

5 Conclusion

We present SKCC, a skill compilation design that achieves portable and secure deployment of agent skills across heterogeneous frameworks. Through a four-phase pipeline centered on SKIR, SKCC decouples skill semantics from framework-specific formatting, enabling skills to be authored once and compiled to diverse targets. A compile-time Security Optimizer enforces safety constraints via Anti-Skill Injection before skills reach any agent’s context window. Experiments across four frameworks

demonstrate consistent pass rate improvements (up to +13.5%), 94.8% Anti-Skill Injection coverage, and 10–46% runtime token savings, confirming that compiler-driven adaptation is both effective and practical for the emerging agent skill ecosystem.

Acknowledgments and Disclosure of Funding

This work is a preprint currently under review. The SKCC project (originally named Nexa-Skill-Compiler, NSC) initiated in March 2026; Core compiler repository open-sourced via GitHub (<https://github.com/Nexa-Language/Skill-Compiler>) on April 3rd, 2026; Initial draft completed in April 2026; Paper submitted in May 2026. We document this timeline to clarify the independent nature of our contributions relative to concurrent developments in the agent skill ecosystem.

This project is part of the Nexa-lang Project. The homepage for SKCC is available at <https://skcc.nexa-lang.com/>. All rights reserved by Nexa-lang Project & arcSYSu Lab, Sun Yat-sen University.

References

- [1] Michael Wooldridge and Nicholas R. Jennings. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995. doi: 10.1017/S0269888900008122.
- [2] Shunyu Yao, Dian Yu, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2305.10601.
- [3] Lei Wang, Chen Ma, et al. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.
- [4] Anthropic. Claude Code Overview, 2026. URL <https://code.claude.com/docs/en/overview>.
- [5] OpenAI. Codex Documentation, 2026. URL <https://developers.openai.com/codex>.
- [6] Google. Gemini CLI Documentation, 2026. URL <https://google-gemini.github.io/gemini-cli/docs/>.
- [7] Kimi. Kimi CLI Documentation, 2026. URL <https://moonshotai.github.io/kimi-cli/en/guides/getting-started.html>.
- [8] Agent Skills. SKILL.md Specification and Progressive Disclosure Mechanism, 2026. URL <https://deepwiki.com/agentskills/agentskills/2.2-skill.md-specification>.
- [9] Renjun Xu, Yang Yan, et al. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward, 2026.
- [10] Anthropic. Anthropic Skills: Public repository for Agent Skills, 2026. URL <https://github.com/anthropics/skills>.
- [11] Affan-m. Everything Claude Code: The agent harness performance optimization system, 2026. URL <https://github.com/affaan-m/everything-claude-code>.
- [12] getSentry. Sentry Skills: Agent Skills used by the Sentry team for development, 2026. URL <https://github.com/getsentry/skills>.
- [13] Jia He, Mukund Rungta, et al. Does Prompt Formatting Have Any Impact on LLM Performance?, 2024.
- [14] Anthropic. Claude API Docs: Prompting Best Practices — Structure Prompts with XML Tags, 2026. URL <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>.
- [15] OpenAI. Structured Outputs and Format Tax Elimination, 2025. URL <https://platform.openai.com/docs/guides/structured-outputs>.

- [16] Improving Agents. Which Nested Data Format Do LLMs Understand Best? JSON vs. YAML vs. XML vs. Markdown, 2025. URL <https://www.improvingagents.com/blog/best-nested-data-format/>.
- [17] Luca Beurer-Kellner, Aleksei Kudrinskii, Marco Milanta, Kristian Bonde Nielsen, Hemang Sarkar, and Liran Tal. Technical report: Exploring the emerging threats of the agent skill ecosystem, 2026. URL <https://arxiv.org/abs/2605.28588>.
- [18] Agent Skills. SKILL.md Explained: How to Structure Your Product for AI Agents — Add Guardrails and Common Pitfalls, 2026. URL <https://www.gitbook.com/blog/skill-md>.
- [19] Anthropic. Model Context Protocol (MCP) Specification, 2025. URL <https://modelcontextprotocol.io/docs/>.
- [20] Alfred V. Aho, Ravi Sethi, et al. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Reading, MA, 1986. ISBN 0-201-10088-6.
- [21] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, San Francisco, CA, 1997. ISBN 1-55860-320-4.
- [22] John Strong, Joseph Wegstein, et al. The Problem of Programming Communication with Changing Machines: A Proposed Solution. *Communications of the ACM*, 1(8):12–18, 1958. doi: 10.1145/368892.368915.
- [23] Chris Lattner and Vikram Adve. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *International Symposium on Code Generation and Optimization (CGO)*, 2004. doi: 10.1109/CGO.2004.1281665.
- [24] Chris Lattner, Mehdi Amini, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *International Symposium on Code Generation and Optimization (CGO)*, 2021. doi: 10.1109/CGO51591.2021.9370308.
- [25] Laszlo Szekeres, Mathias Payer, et al. SoK: Eternal War in Memory. In *IEEE Symposium on Security and Privacy (S&P)*, 2013. doi: 10.1109/SP.2013.13.
- [26] Reddit r/ClaudeAI. Anthropic’s Official Take on XML-Structured Prompting as the Core Strategy, 2026. URL <https://www.reddit.com/r/ClaudeAI/comments/1psxuv7/>.
- [27] Roy Philip. JSON vs. XML: A Data-Driven Analysis of LLM Parsing Efficiency, 2025. URL <https://royphilip.xyz/blog/json-vs-xml-llm-showdown>.
- [28] Steve Kinney. Prompt Engineering Across the OpenAI, Anthropic, and Gemini APIs, 2026. URL <https://stevekinney.com/writing/prompt-engineering-frontier-llms>.
- [29] Yuanye Liu, Jiahang Xu, et al. Beyond Prompt Content: Enhancing LLM Performance via Content-Format Integrated Prompt Optimization, 2025.
- [30] Renxi Wang, Xudong Han, et al. ToolGen: Unified Tool Retrieval and Calling via Generation. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2410.03439.
- [31] Weihang Su, Jianming Long, et al. Skill Retrieval Augmentation for Agentic AI, 2026.
- [32] Darren Edge, Ha Trinh, et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization, 2024.
- [33] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019. doi: 10.18653/v1/D19-1410.
- [34] Yujian Liu, Jiabao Ji, et al. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings, 2026.

- [35] Benjamin Mikek, Danylo Vashchilenko, et al. Agentic Code Optimization via Compiler-LLM Cooperation, 2026.
- [36] Sehoon Kim, Suhong Moon, et al. An LLM Compiler for Parallel Function Calling. In *International Conference on Machine Learning (ICML)*, 2024. doi: 10.48550/arXiv.2312.04511.
- [37] Le Chen, Erhu Feng, et al. SkVM: Revisiting Language VM for Skills Across Heterogeneous LLMs and Harnesses, 2026.
- [38] A. B. V. Kumar. Deep Dive SKILL.md (Part 1/2): Negative Boundaries and Triggering Accuracy, 2026. URL <https://abvijaykumar.medium.com/deep-dive-skill-md-part-1-2-09fc9a536996>.
- [39] Hao Wang, Niels Mündler, et al. SecPI: Secure code generation with reasoning models via security reasoning internalization, 2026.
- [40] Xiangyi Li, Wenbo Chen, et al. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks, 2026.
- [41] NextLevelBuilder. UI/UX Pro Max Skill: An Agent Skill for UI/UX design tasks, 2026. URL <https://github.com/nextlevelbuilder/ui-ux-pro-max-skill>.
- [42] Harbor Framework Team. Harbor: A Framework for Evaluating and Optimizing Agents and Models in Container Environments, 2026. URL <https://github.com/harbor-framework/harbor>.
- [43] Xingyao Wang, Simon Rosenberg, et al. The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents. In *Conference on Machine Learning and Systems (MLSys)*, 2026. doi: 10.48550/arXiv.2511.03690.

A Implementation Details

SKCC is implemented in Rust and organized into four crates:

nexa-skill-cli. CLI entry point using `clap` for argument parsing and `miette` for diagnostic rendering. Provides commands: `build` (compile skills), `check` (validate without emitting), `validate` (strict validation), `init` (scaffold new skill from template), `list` (enumerate skills in directory), `index` (generate routing manifest), and `clean` (remove compiled artifacts).

nexa-skill-core. Core compilation logic organized into six modules: `frontend` (frontmatter parsing, Markdown event-stream parsing, AST construction), `ir` (SKIR definition, IR builder, type mapper, nested data detector), `analyzer` (schema validator, MCP dependency checker, permission auditor, anti-skill injector), `backend` (Emitter trait, EmitterRegistry, four framework-specific Target Emitters, routing manifest generator), `error` (diagnostic types with source spans), and `security` (security baseline, permission types, security level classification).

nexa-skill-templates. Askama template engine with Jinja2-style compile-time-validated templates: `claude_xml.j2` (XML-tagged SKILL.md for Claude), `codex_md.j2` (XML-tagged Markdown for Codex), `gemini_md_v2.j2` (Markdown with conditional YAML for Gemini), and `kimi_md.j2` (full Markdown for Kimi). Each template is paired with a context struct that maps SKIR fields to template variables.

npm-nexa-skill-compiler. npm wrapper package that downloads the precompiled Rust binary and exposes the `nsc` command globally for Node.js users, enabling integration with JavaScript-based agent toolchains.

Key dependencies and design choices.

- `Arc<str>` for zero-copy string sharing across compilation phases and Target Emitters.
- `serde` and `serde_json` for SKIR serialization and JSON Schema handling.
- `serde_yaml` for YAML frontmatter parsing and YAML asset generation.
- `pulldown-cmark` for Markdown event-stream parsing.

- sha2 for source file integrity hashing.
- chrono for compilation timestamp recording.
- askama for compile-time template validation.

Memory optimization. SKIR uses `Arc<str>` for all string fields shared across Target Emitters, enabling zero-copy cloning. The `ValidatedSKIR` wrapper adds only a `Vec<Diagnostic>` without duplicating the underlying IR. For batch compilation of large skill corpora (e.g., 233 skills), the compiler processes skills sequentially with per-skill memory deallocation, keeping peak memory usage below 50MB.

A.1 Compilation Performance Details

On a standard development machine (Intel i9-13900H, 32GB RAM), single-skill compilation (all four targets) completes in under 10ms, with the Security Optimizer phase accounting for approximately 40% of total time. Batch compilation of 225 skills completes in approximately 1.8 seconds (8ms average per skill), demonstrating linear scaling with corpus size. Complexity has minimal impact on compilation time (simple 8.54ms to complex 9.13ms, only +0.59ms), and the maximum compilation time is 22.89ms, well below user perception thresholds.

B Data Validity

All experiments use the SkillsBench benchmark (89 tasks) with Docker-based execution and automated pytest verification. Due to regional API availability constraints and network conditions inherent to cloud-based LLM evaluation, a small number of trials across frameworks produced execution failures (e.g., container timeouts, API rate limits). These failures were strictly due to infrastructure issues and were excluded prior to any analysis of reward outcomes, ensuring unbiased comparison. All reported results are based on paired tasks where both conditions completed successfully.

C Design Artifacts

C.1 Key Insights from Evaluation

Our experiments demonstrate consistent compilation gains across four frameworks, with gains proven model-specific through ablation studies. Engineering metrics confirm compilation latency under 10ms, Anti-Skill Injection coverage of 94.8%, and runtime token savings of 10–46%. Two system-level insights emerge from these results.

Format Tolerance vs. Format Sensitivity. Compilation gains correlate with the underlying model’s format sensitivity. Claude shows the largest improvement ($d = 0.60$) because its training distribution heavily favors XML-tagged inputs; the compiler aligns structural encoding with parsing expectations. Gemini shows minimal reward improvement ($d \approx 0$) because it is relatively format-tolerant. This validates SKCC’s core premise: different models have different format preferences, and a one-size-fits-all SKILL.md inevitably underperforms on format-sensitive frameworks.

Static Overhead vs. Dynamic Efficiency. Compilation increases static skill size by 4–25% yet reduces dynamic token consumption by 10–46% during execution. Structured formats serve as cognitive scaffolding, reducing parsing ambiguity and trial-and-error. The compiler invests tokens upfront in structural clarity, which the model repays through more efficient execution. The true value of skill compilation lies not in compression but in structural investment: spending tokens on clarity to save tokens on execution.

C.2 Framework-Specific Emission Details

The following describes the format hardening strategy for each target framework, as referenced in §3.3.

Claude (XML Semantic Layering). Leveraging Anthropic’s documented preference for XML-tagged prompts [14], this target wraps all structural elements in semantic XML tags: procedures in `<execution_steps>/<step>` with `order` and `critical` attributes, constraints in

<strict_constraints>/<anti_pattern>, and examples in <examples>/<example> with nested <input> and <output>. This semantic layering reduces misinterpretation and improves reasoning accuracy by up to 23%.

Codex (XML-Tagged Markdown). This target produces a hybrid XML-tagged Markdown format: instructions in <skill>/<instructions>, constraints in <constraints>/<forbidden>, and examples in <examples>/<example>. This provides structural markers for parsing while avoiding the JSON “format tax” that degrades GPT-series model performance [15]. Structured output enforcement is delegated to the OpenAI API’s Structured Outputs feature, decoupling reasoning from formatting.

Gemini (Markdown + Conditional YAML). Applying the nested data detection flag from the IR Builder phase, this target conditionally renders deeply nested schemas (depth ≥ 3) as YAML code blocks while keeping shallow structures in standard Markdown. When YAML optimization is triggered, separate YAML asset files are generated for complex nested structures. This adaptive strategy leverages YAML’s superior parsing accuracy (51.9% vs JSON’s 43.1%) for nested data while avoiding unnecessary format switching for simple structures.

Kimi (Full Markdown Preservation). This target preserves all skill details in comprehensive Markdown without simplification or format optimization, leveraging Kimi’s ultra-long context window capability. No YAML optimization or content truncation is applied, ensuring maximum information fidelity for frameworks that can process full skill content without token budget constraints.

C.3 SKIR Example

Listing 1 shows a simplified SKIR instance for a “github-api-client” skill, illustrating how the raw Markdown source is normalized into a structured, framework-agnostic representation.

Listing 1: Simplified SKIR for a “github-api-client” skill. Note the `anti_skill_constraints` field, which was automatically injected by the Security Optimizer, and the structured procedures array.

```
1 {
2   "name": "github-api-client",
3   "version": "1.0.0",
4   "description": "Interact with GitHub REST API",
5   "mcp_servers": ["github-mcp"],
6   "input_schema": {
7     "type": "object",
8     "properties": {
9       "repo": { "type": "string" },
10      "action": { "type": "string",
11        "enum": ["create_issue", "list_prs"] }
12    }
13  },
14  "security_level": "high",
15  "hitl_required": true,
16  "permissions": [
17    { "kind": "network",
18      "scope": "https://api.github.com/*",
19      "read_only": false }
20  ],
21  "procedures": [
22    { "order": 1,
23      "instruction": "Validate GitHub token from env",
24      "is_critical": true },
25    { "order": 2,
26      "instruction": "Construct REST request" },
27    { "order": 3,
28      "instruction": "Execute HTTP POST to GitHub API" }
29  ],
30  "anti_skill_constraints": [
31    {
32      "source": "anti-skill-injector",
33      "content": "Never execute HTTP without timeout...",
```

```

34     "level": "warning",
35     "scope": "global"
36   }
37 ],
38 "requires_yaml_optimization": false,
39 "mode": "sequential"
40 }

```

C.4 Anti-Skill Injection Rules

Table 6: Anti-Skill Injection Rules

Anti-Pattern	Trigger Keywords	Injected Constraint
HTTP safety	HTTP, GET, POST, fetch, request	Never execute HTTP without timeout (10s). Max 3 retries on 403.
HTML Parse safety	BeautifulSoup, HTML parse, scrape	Do not parse raw JS variables with HTML parsers. Fallback to Regex.
Destructive DB safety	DROP, DELETE, TRUNCATE	No destructive DB ops without user confirmation. Show affected rows.
Loop safety	while, loop, repeat	All loops must have max iteration limit (1000).

The four rules were selected based on common vulnerability patterns observed in community skill audits [17]. HTTP safety is the most frequently triggered rule (91.4%) because virtually all skills that interact with external APIs contain HTTP-related procedure steps. The injection mechanism is designed to be extensible: new rules can be added by defining a trigger pattern and a corresponding constraint template, without modifying the compilation pipeline.

C.5 Four-Framework Format Divergence

Listing 2 illustrates the format divergence across Target Emitters for a single SKIR.

Listing 2: Format divergence across four Target Emitters for a single SKIR. Note the Gemini Target Emitter’s conditional YAML rendering (triggered by nesting depth ≥ 3) and the consistent presence of anti-skill constraints across all formats.

```

1  \SkIR{} (framework-independent)
2  |-- name: "data-migration"
3  |-- procedures: [3 steps]
4  |-- input_schema: { nested depth = 4 }
5  +-- anti_skill_constraints: [1 HTTP safety]
6
7  Compiled Outputs:
8
9  Claude:  <agent_skill>
10           <execution_steps>
11             <step order="1" critical="true">...</step>
12           </execution_steps>
13           <strict_constraints>
14             <anti_pattern source="anti-skill-injector">
15               ...
16             </anti_pattern>
17           </strict_constraints>
18         </agent_skill>
19
20  Codex:   <skill name="data-migration">
21           <instructions>...</instructions>
22           <constraints>
23             <forbidden>...</forbidden>
24           </constraints>
25         </skill>
26
27  Gemini:   # data-migration
28           ## Procedures
29           1. ... **[CRITICAL]**
30           ## Parameter Schema (YAML Optimized)

```



```

31     '''yaml
32     type: object
33     properties:
34         migration_config:
35             type: object
36             properties:
37                 source_db:
38                     type: object
39                     properties:
40                         host: { type: string }
41     '''
42
43     Kimi:      # data-migration
44               ## Description
45               ...
46               ## Procedures
47               1. ... **[CRITICAL]**
48               ## Parameter Schema
49               - 'migration_config.source_db.host' (string): ...

```

D Complete Experimental Data

D.1 Four-Model Comparison Summary

Table 7: Four-Model Comparison Summary

Model	Paired	Δ Rwd.	p	d	Verdict
claude-opus-4-6	22-27	+0.26-0.27	0.0096**	0.59-0.60	SKCC $\gg$ Baseline
kimi-k2.5	74	+0.142	0.0063**	0.33	SKCC $>$ Baseline
gpt-5.3-codex	26	+0.067	—	—	SKCC $>$ Baseline
gemini-2.5-pro	18	+0.019	—	—	SKCC $>$ Baseline

The effect sizes follow a clear pattern: Claude shows a medium-to-large effect ($d \approx 0.60$), Kimi shows a small-to-medium effect ($d = 0.33$), and Codex and Gemini show negligible effects. This gradient aligns with the degree of format sensitivity documented for each model in prior work [13, 14, 16]. The statistical significance on Claude and Kimi (both $p < 0.01$) is particularly noteworthy given the modest sample sizes, indicating that the compilation effect is robust even under conservative testing.

D.2 Claude Code — Complete Data

Table 8: Claude Code — Complete Paired Statistical Tests

Cmp.	n	Mean Δ	W/T/L	t	p	d
SKCC vs V	23	+0.265	7/16/0	2.837	0.0096**	0.592
SKCC vs Baseline	22	+0.274	7/15/0	2.820	0.0103*	0.601
Baseline vs V	26	+0.002	3/21/2	0.031	0.9756	0.006

Task classification (22 paired SKCC vs Baseline): SKCC Better: 7 tasks (31.8%) — 6 flipped from reward=0 to reward=1; SKCC Worse: 0 tasks (0%); Tie: 15 tasks (68.2%).

The Claude results are particularly compelling because SKCC never underperforms Baseline in any paired comparison (0 losses). The 6 complete flips from failure to success demonstrate that XML Semantic Layering addresses a genuine parsing failure mode: these are tasks where Claude could not interpret the plain Markdown instructions at all, but succeeded when the same semantic content was presented in its native XML format.

D.3 Kimi CLI — Complete Data

Table 9: Kimi CLI — Complete Statistical Tests

Test	Statistic	p	Sig.
Paired t-test	$t = 2.815$	0.0063	$p < 0.01$
Wilcoxon signed-rank	$W = 22.0$	0.0050	$p < 0.01$
Non-tie only ($n = 17$)	$t = 3.449$	0.0033	$p < 0.01$
Cohen’s d (paired)	0.327	—	Small effect

Task classification (74 paired): SKCC Better: 13 discriminative wins (81.25%); SKCC Worse: 3 (18.75%); Tie: 58 (78.4%); 13 tasks flipped from reward=0 to reward=1.

Kimi achieves the largest sample size (74 paired tasks) and the strongest statistical significance ($p = 0.0063$). The consistency across parametric (t-test) and non-parametric (Wilcoxon) tests confirms that the effect is not driven by distributional assumptions. The 13 task flips from failure to success represent a substantial practical improvement: nearly one in five originally failing tasks becomes solvable after compilation.

D.4 Ablation Study — Full Data

Table 10: Ablation Study — Complete Metrics

Model	Framework	Backend	Succ. (Baseline/SKCC)	Paired	Rwd. (Baseline/SKCC)	p	d	Eff.
kimi-k2.5	Kimi CLI	Kimi	26/75 → 36/76	74	0.341 → 0.483	0.0063	+0.33	SKCC > Baseline
glm-5.1	OpenHands	Kimi	43/88 → 44/88	32	—	0.857	−0.03	SKCC ≈ Baseline
deepseek-v4-flash	OpenHands	Kimi	64/88 → 65/88	50	—	0.2561	−0.14	Baseline > SKCC

The ablation results reveal a critical property of format optimization: the same compiled output that benefits one model can be neutral or even slightly harmful to another. This asymmetry is the empirical foundation for SKCC’s multi-backend architecture. If a single optimal format existed across all models, a one-time conversion would suffice; the fact that Kimi’s optimal format underperforms on DeepSeek demonstrates that per-model emission is necessary.

D.5 Ablation Radar Chart

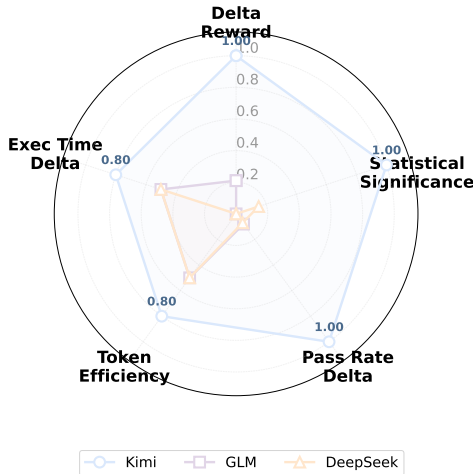


Figure 6: Ablation study radar chart: the same Kimi-compiled format produces divergent effects across three models across five evaluation dimensions. Kimi shows strong gains on all axes, while GLM and DeepSeek show flat or negative effects, confirming model-specificity of compilation gains.

The radar chart visualizes the ablation results across five normalized dimensions: Delta Reward, Statistical Significance, Pass Rate Delta, Token Efficiency, and Execution Time Delta. Kimi dominates across all dimensions, while GLM and DeepSeek cluster near zero. The stark visual separation reinforces the core finding: compilation gains are not universal but depend on the alignment between the compiled format and the target model’s training distribution.

D.6 Expansion Overhead by Complexity

Table 11: Expansion Overhead by Complexity

Complexity	Claude Ovhd.	Kimi Ovhd.	Claude w/ Reduction	Kimi w/ Reduction
Simple (avg 298t)	+95.0%	+37.4%	0/8 (0%)	0/8 (0%)
Medium (avg 819t)	+43.0%	+14.6%	4/74 (5.4%)	36/74 (48.6%)
Complex (avg 2765t)	+11.4%	−3.1%	31/143 (21.7%)	101/143 (70.1%)

The expansion overhead exhibits an inverse relationship with skill complexity: simple skills incur the largest relative overhead because the fixed cost of XML tags and constraints dominates, while complex skills see diminishing overhead as the fixed cost is amortized over a larger base. For Kimi, complex skills actually shrink after compilation (−3.1%) because the structured formatting eliminates redundant Markdown boilerplate. The “w/ Reduction” columns count skills where the compiled output is smaller than the original, showing that this effect becomes common for complex skills (70.1% on Kimi).

D.7 Claude Code — Full Token Consumption

Table 12: Claude Code — Full Token Consumption Comparison

Condition	Succ. Tasks	Input T.	Output T.	Cache T.	Total	Task Avg.
Vanilla	34	19.5M	421K	17.2M	~19.9M	~0.59M
Baseline	40	32.9M	574K	30.1M	~33.4M	~0.84M
SKCC	29	18.3M	459K	15.8M	~18.7M	~0.65M

The token consumption data reveals an important pattern: Baseline consumes more tokens than Vanilla (no skill) because the format-agnostic Markdown adds content without improving structure, leading to more trial-and-error. SKCC reverses this: despite adding XML tags and constraints, the clearer structure reduces both input tokens (18.3M vs. 32.9M) and output tokens (459K vs. 574K), indicating that the model requires fewer reasoning steps when instructions are presented in its preferred format.

D.8 Anti-Skill Injection — Full Statistics

Table 13: Anti-Skill Trigger Statistics (233 skills)

Metric	Value
Total skills	233
Skills triggering Anti-Skill	221 (94.8%)
Skills not triggering	12 (5.2%)

The near-universal trigger rate (94.8%) underscores the prevalence of potentially dangerous patterns in community skills. The 12 skills that did not trigger any rule were predominantly simple utility skills with no external interactions (e.g., string formatting helpers). This suggests that Anti-Skill Injection provides meaningful coverage for virtually all skills that perform substantive operations.

D.9 Rule Trigger Distribution — Full Data

Table 14: Rule Trigger Distribution (Full)

Anti-Skill Rule	Triggered	Keywords	Example Constraint
HTTP safety	212 (91.4%)	HTTP, GET, POST, fetch, request	Timeout (10s), max 3 retries on 403
Loop safety	104 (44.6%)	while, loop, repeat	Max iteration limit (1000)
DB safety	78 (33.5%)	DROP, DELETE, TRUNCATE	No destructive ops without confirmation
Parse safety	2 (0.9%)	BeautifulSoup, HTML parse, scrape	No parsing raw JS with HTML parsers

The distribution reveals that HTTP and loop safety dominate, reflecting the fact that most agent skills involve API calls and iterative processing. DB safety triggers on approximately one-third of skills, consistent with the prevalence of data manipulation tasks. Parse safety is rarely triggered because few skills perform HTML scraping; this rule could be expanded to cover additional parsing scenarios (e.g., JSON parsing without schema validation) in future work.

D.10 Compilation Interception Types

Table 15: Compilation Interception Types

Interception Type	Cnt.	Description	Example Skills
YAML format violation	5	Frontend rejected non-standard frontmatter	senior-java, senior-data-engineer, threejs (×2), data-reconciliation
Security check interception	4	Dangerous operations or sensitive content	ssh-penetration-testing, restclient-migration, jakarta-namespace, spring-security-6
Schema validation interception	1	IR builder found illegal field types	nlp-research-repo-package-installment

The interception types illustrate SKCC’s defense-in-depth approach: YAML format violations are caught at the parsing stage, security interceptions at the analysis stage, and schema violations at the IR construction stage. Each layer catches a distinct class of problems, and no single layer could catch all three. The 95.7% successful compilation rate (221/231) demonstrates that the interception criteria are appropriately calibrated: they block genuinely problematic skills without being overly restrictive.

E Limitations

Scope of Evaluated Frameworks. Our evaluation covers four mainstream agent frameworks (Claude Code, Codex CLI, Gemini CLI, Kimi CLI). While these represent the most widely used systems at the time of writing, the agent ecosystem is rapidly evolving, and new frameworks may exhibit different format sensitivities. The polymorphic Emitter architecture is designed to accommodate new targets, but empirical validation on additional frameworks remains future work.

Anti-Skill Rule Coverage. The current Anti-Skill Injection rules target four common vulnerability classes. While these cover the most prevalent patterns identified in community audits [17], they do not exhaust the space of possible security issues. In particular, prompt injection attacks, data exfiltration through side channels, and supply chain vulnerabilities in MCP dependencies are not addressed by the current rule set.

Benchmark Scope. SkillsBench provides 89 tasks spanning programming and data analysis domains. While this is a substantial benchmark for skill evaluation, it does not cover all agent use cases (e.g., creative writing, conversational tasks, multi-agent coordination). The compilation gains we observe may not generalize to domains where format sensitivity plays a different role.

Compilation Granularity. SKCC operates at the granularity of entire SKILL.md files. Finer-grained compilation (e.g., per-procedure or per-example optimization) could yield additional gains but would require more sophisticated dependency analysis. This is a direction for future work.

F Broader Impact

SKCC aims to improve the reliability and security of LLM-based agent systems. By making skills portable across frameworks, it reduces the barrier to entry for skill authors and lowers the maintenance

burden for skill consumers. By enforcing security constraints at compile time, it provides a systematic defense against vulnerabilities that currently rely on author diligence alone.

Positive Impacts. The primary positive impact is improved agent safety: compile-time security hardening prevents dangerous skills from reaching agent context windows, reducing the risk of unintended harmful actions. The portability mechanism also promotes an open skill ecosystem where authors can write once and deploy anywhere, potentially accelerating the development of high-quality, reusable agent capabilities.

Potential Negative Impacts. Compile-time security analysis can produce false positives that block legitimate skills, potentially frustrating developers. The current interception rate (4.3%) is low, but as rule sets expand, calibration will be important to maintain usability. Additionally, the existence of a compilation framework could create a false sense of security: SKCC addresses format-level and pattern-level vulnerabilities but cannot guarantee semantic safety of skill logic, which ultimately depends on the skill author’s intent and the agent’s runtime behavior.

Mitigation Strategies. SKCC’s diagnostic system provides non-blocking warnings for uncertain cases, allowing developers to review and override automated decisions. The tiered security classification enables graduated enforcement rather than binary accept/reject decisions. Future work on explainable security analysis could further improve developer trust and reduce false positive friction.

G How to Cite

If you find SKCC useful for your research, please consider citing:

```
@misc{ouyang2026skcc,
  title={SkCC: Portable and Secure Skill Compilation
        for Cross-Framework LLM Agents},
  author={Yipeng Ouyang and Yi Xiao and Yuhao Gu
        and Xianwei Zhang},
  year={2026},
  eprint={2605.03353},
  archivePrefix={arXiv},
  url={https://arxiv.org/abs/2605.03353},
}
```